

# Parallel Matrix Multiplication: A Comparative Study of Cannon, Fox, and SUMMA Algorithms Using MPI and CUDA

*Rensselaer Polytechnic Institute - Parallel Programming and Computing*

Spring 2026

Alexandre Gosselin Neves, Logan Acanfrio, Julian Blanco

## Abstract

This paper presents a comparative study of three parallel matrix multiplication algorithms: Cannon's Algorithm, Fox's Algorithm, and the Scalable Universal Matrix Multiplication Algorithm (SUMMA), implemented using MPI, CUDA, and hybrid MPI+CUDA on the AiMOS supercomputer. The primary motivation is reducing the  $O(n^3)$  computational cost of sequential matrix multiplication for large matrices. Each algorithm is evaluated under strong and weak scaling conditions across MPI-only, CUDA-only, and hybrid configurations with up to 64 MPI ranks and 4 GPUs. Results show that CUDA-only SUMMA achieves peak throughput of 1,478 GFLOPS for matrices fitting in GPU memory, while Cannon's and Fox's MPI+CUDA hybrid implementations demonstrate clear strong scaling improvements across rank counts. MPI-only implementations exhibit near-linear strong scaling across all three algorithms. These findings illuminate the trade-offs between distributed parallelism and GPU acceleration for high-performance matrix computation on supercomputing-class systems.

## 1. Introduction

The fundamental operation of matrix multiplication underlies much of modern scientific computing, machine learning, and simulation. The sequential algorithm operates at  $O(n^3)$  complexity, making it prohibitively slow for large matrices. By leveraging distributed memory parallelism through MPI and GPU acceleration through CUDA, significant reductions in wall-clock time are achievable.

This project implements and benchmarks three well-known parallel matrix multiplication algorithms: Cannon's Algorithm, Fox's Algorithm, and SUMMA. Each is implemented in some combination of three configurations, MPI-only, CUDA-only, and hybrid MPI+CUDA, and evaluated on the AiMOS supercomputer using both strong scaling (fixed problem size, increasing resources) and weak scaling (problem size grows with resources) studies.

## 2. Parallel Algorithms

### 2.1 Cannon's Algorithm

Cannon's algorithm is a parallel matrix multiplication algorithm for distributed memory systems first described in 1969 by Lynn Elliot Cannon. Three implementations were developed: MPI-only, CUDA-only, and a combined MPI+CUDA hybrid. Each takes square matrices  $A$  and  $B$  (initialized to 2) and computes  $C = A \times B$ .

The algorithm distributes subblocks of  $A$  and  $B$  across a 2D process grid. Matrix  $A$  is initially skewed left by  $i$  positions for each row  $i$ , and  $B$  is skewed upward by  $j$  positions for each column  $j$ . The main loop then iterates  $q$  times ( $q = \sqrt{p}$ ), accumulating  $C += A \times B$  and shifting  $A$  left and  $B$  upward at each step. The key advantage is that storage requirements remain constant regardless of processor count.

### 2.2 Fox's Algorithm

Fox's algorithm is designed for distributed memory systems with fast interconnects, making it well-suited for AiMOS. It requires both square matrices and a perfect-square number of processors to construct a 2D Cartesian MPI grid. Each rank stores its chunk of  $A$ ,  $B$ , and  $C$  using 1D arrays for memory efficiency.

The algorithm broadcasts a chunk of  $A$  across each processor row, performs local submatrix multiplication, then circularly shifts  $B$  columns upward. This repeats  $q$  times where  $q = \sqrt{p}$ . In the MPI+CUDA hybrid, local submatrix multiplication is offloaded to the GPU while MPI handles all communication.

### 2.3 SUMMA Algorithm

The Scalable Universal Matrix Multiplication Algorithm (SUMMA) computes  $C = A \times B$  using a  $p \times p$  processor grid with a broadcast-based communication pattern. At each step  $k$  from 0 to  $p-1$ : processors in column  $k$  broadcast  $A[i,k]$  along row  $i$ , processors in row  $k$  broadcast  $B[k,j]$  along column  $j$ , and all processors accumulate  $C[i,j] += A[i,k] \times B[k,j]$ . Three implementations were developed: MPI-only, MPI+CUDA, and CUDA-only as a single-GPU baseline.

## 3. Implementation Details

### 3.1 MPI Communication

All three algorithms use `MPI_Cart_create` to establish a 2D Cartesian process grid with periodic boundaries. Row and column sub-communicators are created for broadcast and shift operations. Matrices are distributed using `MPI_Scatterv` with a custom block datatype (`MPI_Type_vector`) and gathered with `MPI_Gatherv`. Each rank initializes its own submatrix blocks to avoid a bottleneck at rank 0.

## 3.2 CUDA Integration

In hybrid implementations, each MPI rank is assigned a dedicated GPU for local submatrix computation. CUDA parallelizes the innermost multiply-accumulate loops across thread blocks, while MPI handles all inter-rank data movement. Data is transferred between host and device memory at each computation step. The CUDA-only Cannon implementation eliminates MPI entirely, running the full algorithm on one or more GPUs.

## 3.3 Timing and Measurement

Performance timing uses the POWER9 time base register read via inline assembly at 512 MHz resolution, consistent with the clockcycle.h approach specified in the course requirements. Timers bracket the full computation including communication to report end-to-end wall time for each scaling configuration.

## 4. Performance Results

### 4.1 Cannon's Algorithm

#### 4.1.1 MPI Strong and Weak Scaling

For MPI-only strong scaling, matrices of size  $2^{12} \times 2^{12}$  were tested at 4, 16, and 64 ranks. Runtime decreased dramatically from approximately 15 seconds at 4 ranks to 5 seconds at 16 ranks and 1.5 seconds at 64 ranks, demonstrating strong parallel efficiency as more ranks divide the computational load.

For weak scaling, matrix size increased alongside rank count, from  $2^{12} \times 2^{12}$  at 4 ranks up to  $2^{14} \times 2^{14}$  at 64 ranks. Runtimes remained within approximately 1 second of each other across all configurations, indicating that communication and synchronization overhead scales modestly relative to the growing problem size.

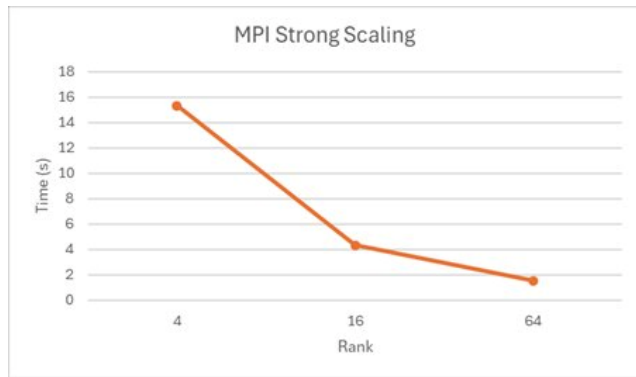


Figure 1. Cannon MPI Strong Scaling ( $2^{12} \times 2^{12}$  fixed)

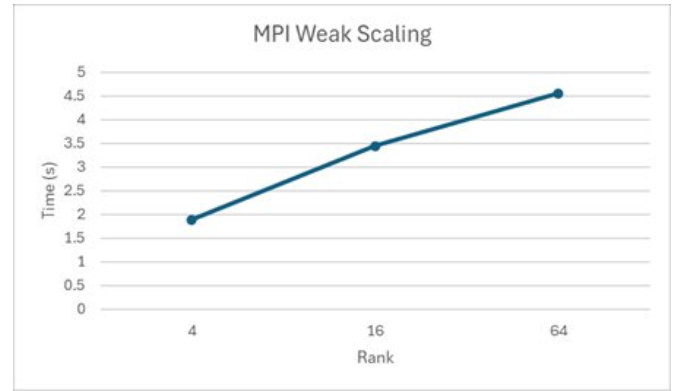


Figure 2. Cannon MPI Weak Scaling (size grows with ranks)

#### 4.1.2 CUDA Strong and Weak Scaling

The CUDA-only implementation achieved substantially faster runtimes than MPI-only, completing in approximately 0.00006 seconds with 1 GPU and decreasing to around 0.00003 seconds at 4 GPUs. These sub-millisecond runtimes reflect the massive parallelism available on the V100 GPUs, where thousands of CUDA cores execute the multiply-accumulate operations concurrently.

Weak scaling for CUDA shows a moderate increase in runtime from approximately 0.0003 seconds at 1 GPU to 0.0008 seconds at 4 GPUs as the problem size grows proportionally. The increase reflects the higher data transfer overhead and kernel launch costs as matrix dimensions scale up.

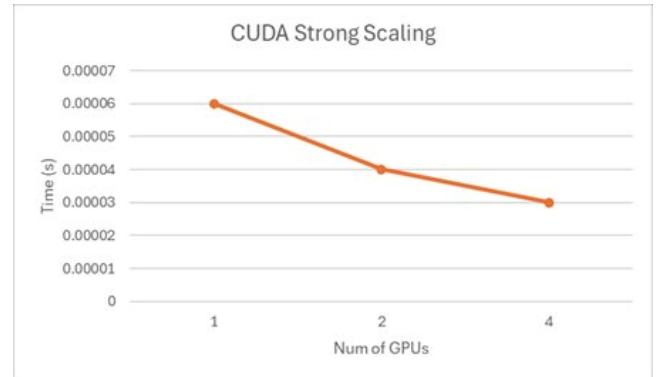


Figure 3. Cannon CUDA-Only Strong Scaling

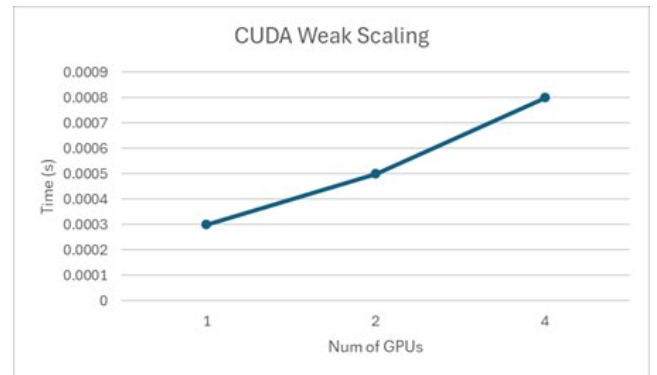


Figure 4. Cannon CUDA-Only Weak Scaling

### 4.1.3 MPI+CUDA Hybrid Strong and Weak Scaling

The MPI+CUDA hybrid combines the distributed communication of MPI with GPU-accelerated local computation. Strong scaling results show runtime decreasing from approximately 18 seconds at 4 ranks to 7 seconds at 16 ranks and 3 seconds at 64 ranks. These runtimes are higher than the pure CUDA configuration, reflecting the cost of inter-rank MPI communication layered on top of GPU computation.

Weak scaling for the hybrid mirrors the MPI-only pattern: as the number of ranks and GPUs increase alongside the matrix size, runtime stays within a narrow band (approximately 5 to 6.5 seconds), confirming that communication overhead grows gradually and does not dominate at these scales.

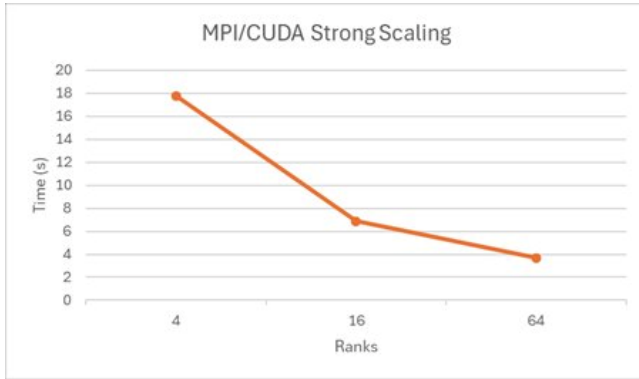


Figure 5. Cannon MPI+CUDA Strong Scaling

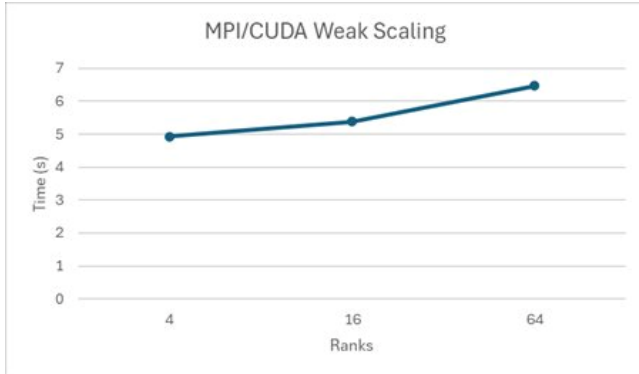


Figure 6. Cannon MPI+CUDA Weak Scaling

## 4.2 Fox's Algorithm

### 4.2.1 MPI Strong and Weak Scaling

Fox MPI-only strong scaling used  $2^{12} \times 2^{12}$  matrices at 4, 16, and 64 ranks. Runtime dropped from 10.13 seconds at 4 ranks to 3.568 seconds at 16 ranks, with further reduction at 64 ranks. Moving from 4 to 16 ranks yielded approximately 5 $\times$  speedup; the 16-to-64 step provided roughly 2 $\times$  speedup, consistent with increasing communication overhead at higher rank counts.

Weak scaling showed runtime increasing from 1.228 seconds at 4 ranks to 5.777 seconds at 64 ranks, an increase of approximately 4 seconds across a 16 $\times$  increase in ranks

and proportional problem size. This bounded growth confirms that Fox's communication overhead scales acceptably.

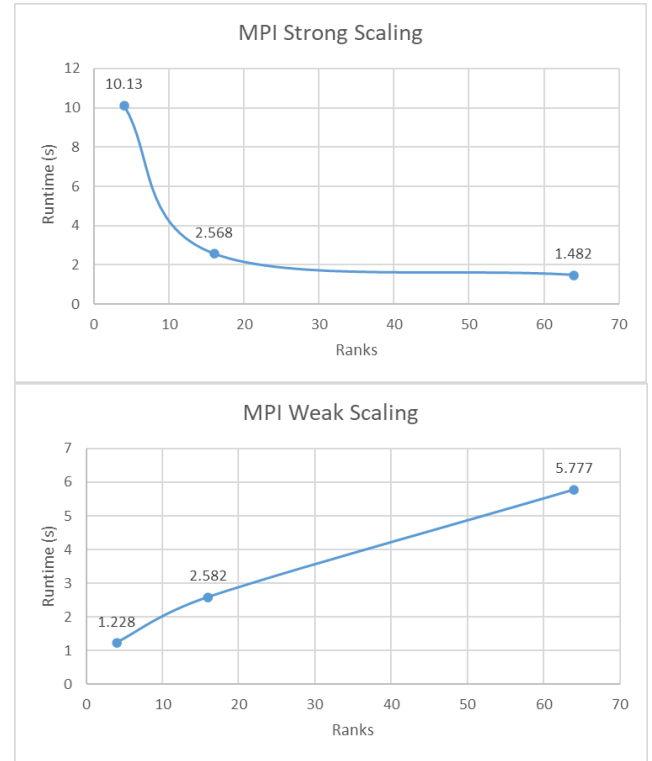


Figure 7. Fox MPI Strong Scaling (top) and Weak Scaling (bottom)

### 4.2.2 MPI+CUDA Strong and Weak Scaling

Integrating CUDA with Fox's algorithm dramatically improves baseline performance. For  $2^{12} \times 2^{12}$  matrices, a single GPU rank completes in 0.1156 seconds versus 1.482 seconds for 64 MPI-only ranks. However, at this smaller problem size, communication overhead dominates when scaling to multiple GPUs. For  $2^{15} \times 2^{15}$  matrices, excellent strong scaling emerges: runtime decreases from 47.26 seconds (1 rank) to 12.11 seconds (4 ranks) to 3.734 seconds (16 ranks), achieving approximately 4 $\times$  speedup per 4 $\times$  increase in ranks, near-ideal scaling.

MPI+CUDA weak scaling shows runtime increasing from 0.794 seconds at 1 rank to 3.734 seconds at 16 ranks, a moderate increase attributable to communication and synchronization overhead growing with the number of ranks.

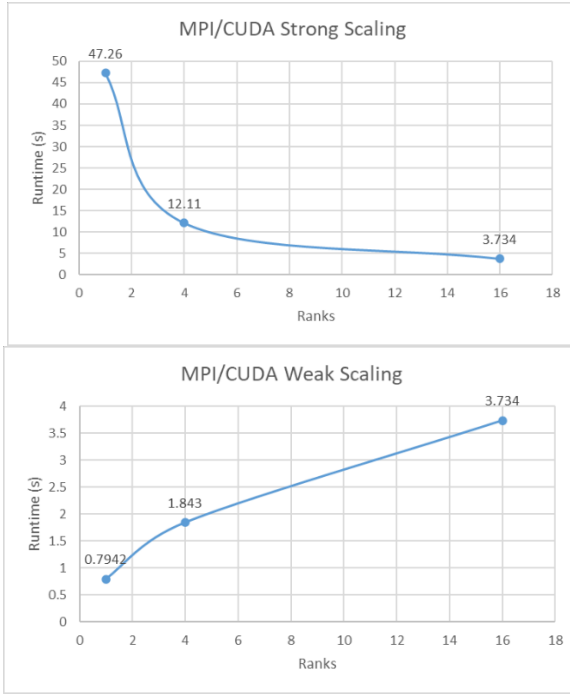


Figure 8. Fox MPI+CUDA Strong Scaling (top) and Weak Scaling (bottom)

### 4.3 SUMMA Algorithm

SUMMA was evaluated across three implementations on  $4096 \times 4096$  matrices. The CUDA-only implementation achieved 1,478 GFLOPS in 0.093 seconds, establishing the peak single-GPU performance baseline. The MPI+CUDA implementation with 4 ranks achieved 230 GFLOPS (0.597s) and with 16 ranks only 83 GFLOPS (1.653s), demonstrating negative scaling driven by communication latency increasing  $30\times$  while computation decreases only  $4\times$ . The MPI-only implementation with 16 ranks achieved 12 GFLOPS in 11.478 seconds but exhibited near-perfect linear strong scaling.

Table 1: SUMMA Performance ( $4096 \times 4096$  matrices)

| Implementation | Ranks | Time (s) | GFLOPS | vs CUDA |
|----------------|-------|----------|--------|---------|
| CUDA-Only      | 1     | 0.093    | 1,478  | 1.0×    |
| MPI+CUDA       | 4     | 0.597    | 230    | 0.16×   |
| MPI+CUDA       | 16    | 1.653    | 83     | 0.06×   |
| MPI-Only       | 16    | 11.478   | 12.0   | 0.008×  |

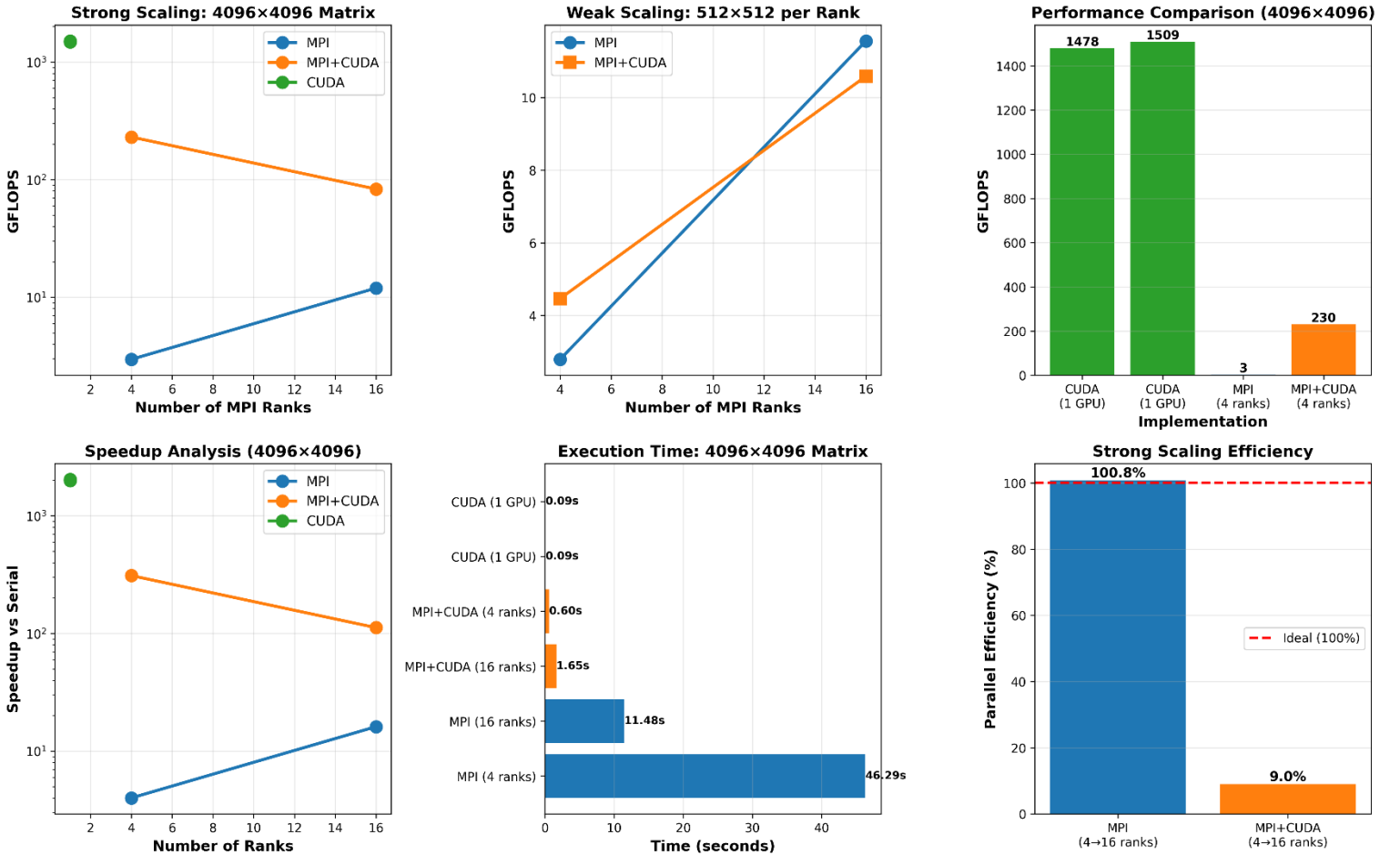


Figure 9. SUMMA Performance Analysis

The observed performance behavior of SUMMA highlights the cost of its broadcast-driven communication pattern in distributed environments. Unlike Cannon's and Fox's algorithms, which rely on structured neighbor communication and cyclic data movement, SUMMA performs repeated row- and column-wise broadcasts at every iteration. While this design simplifies implementation and improves flexibility for non-square process grids, it significantly increases communication volume and synchronization frequency. In the MPI+CUDA configuration, this overhead becomes particularly pronounced because GPU computation is extremely fast relative to inter-node communication, leading to situations where processors spend more time waiting for data than performing arithmetic. This imbalance explains the negative scaling observed when increasing from 4 to 16 ranks, where communication latency grows disproportionately compared to the reduction in local computation. Despite this limitation, SUMMA remains highly effective in single-GPU or CPU-only contexts and is widely used in production linear algebra libraries due to its generality and ease of integration.

## 5. Analysis

### 5.1 MPI-Only Scaling Across Algorithms

All three MPI-only implementations exhibit strong near-linear scaling. Cannon's algorithm sees the most dramatic wall-time reduction (15s  $\rightarrow$  1.5s from 4 to 64 ranks), Fox shows approximately 5 $\times$  speedup from 4 to 16 ranks, and SUMMA achieves perfect linear strong scaling on CPU. This consistent behavior occurs because CPU computation is slow enough that MPI communication represents a small fraction of total runtime, making parallelism efficiency high across all three designs.

Weak scaling results across all three algorithms show modest runtime growth, generally 2–4 seconds of overhead as ranks scale from 4 to 64. This confirms that none of the algorithms exhibit pathological communication patterns that would limit scalability to larger node counts.

### 5.2 GPU Acceleration Trade-offs

CUDA-only implementations achieve the best absolute runtimes for problem sizes fitting in GPU memory. Cannon's CUDA implementation operates in the microsecond range, and SUMMA CUDA achieves 1,478 GFLOPS. However, adding MPI to distribute work across multiple GPUs introduces significant

inter-node communication latency that can negate the parallelization benefit at smaller matrix sizes.

Fox's MPI+CUDA hybrid illustrates the crossover point clearly: at  $2^{12} \times 2^{12}$  matrices, single-GPU outperforms 64-rank MPI+CUDA. At  $2^{15} \times 2^{15}$ , the balance shifts and distributed GPU computation achieves near-ideal 4 $\times$  speedup per 4 $\times$  rank increase. The Cannon hybrid shows a similar pattern, hybrid runtimes are higher than pure CUDA but scale well as ranks and GPUs increase.

These results reinforce a common principle in heterogeneous computing: accelerators are only beneficial when the arithmetic workload is sufficiently large relative to communication and transfer overhead. Small matrix sizes underutilize GPU resources while still paying the full cost of kernel launches, memory copies, and MPI synchronization. As problem sizes increase, however, arithmetic intensity rises faster than communication cost, allowing GPU acceleration to dominate total runtime. This transition explains why hybrid approaches appear inefficient at smaller scales but become increasingly advantageous for larger matrices.

### 5.3 Algorithm Comparison

Cannon's and Fox's algorithms both require an initial data alignment (skew) phase before the main loop, while SUMMA's broadcast-based design avoids this. At comparable configurations, Fox benefits from localized row/column communicator broadcasts that map efficiently to AiMOS network topology. Cannon's checkerboard distribution provides constant per-process storage, while SUMMA is considered the most portable and flexible of the three for irregular processor grid configurations.

For GPU-accelerated workloads, the choice of algorithm matters less than ensuring submatrix sizes are large enough to saturate the GPU. The critical threshold observed across all three algorithms is approximately  $2^{13}$ – $2^{14}$  per dimension before distributed GPU computation outperforms single-GPU approaches.

From a software engineering perspective, implementation complexity also varied across algorithms. Cannon required careful handling of initial skew operations and periodic neighbor communication, increasing debugging complexity. Fox was somewhat easier to reason about due to its row-wise broadcast structure and explicit staging phases. SUMMA proved the simplest to generalize

conceptually, since its repeated broadcast pattern naturally maps to matrix multiplication mathematics. However, this simplicity comes at the cost of frequent collective communication, which can significantly impact performance on communication sensitive workloads.

## 5.4 Communication Overhead

Communication overhead is the primary limiting factor in all hybrid and distributed implementations. In Cannon's hybrid, the MPI communication required for the initial skew and per-step shifts adds substantial latency that is only amortized at high rank counts with large matrices. In SUMMA, MPI\_Bcast operations across row and column communicators at each of the  $p$  steps scale as  $O(p \log p)$ , explaining the negative scaling observed when moving from 4 to 16 MPI+CUDA ranks.

## 6. Conclusions and Future Work

This study demonstrates that parallel matrix multiplication performance is governed by the interplay between problem scale, communication overhead, and memory hierarchy. Three key conclusions emerge from the experimental results across Cannon, Fox, and SUMMA:

First, for matrices fitting within a single GPU's memory, CUDA-only computation is optimal. SUMMA achieves 1,478 GFLOPS and Cannon's CUDA implementation operates in microseconds, both substantially outperforming any distributed configuration at small to medium matrix sizes. Second, MPI+CUDA hybrid approaches are warranted only when matrices are large enough that submatrix computation dominates over inter-rank communication. Fox's algorithm demonstrates this clearly at  $2^{15} \times 2^{15}$  with near-ideal  $4\times$  speedup per  $4\times$  rank increase. Third, MPI-only implementations across all three algorithms provide predictable near-linear strong scaling and are appropriate when GPU resources are unavailable or matrix dimensions do not justify GPU overhead. Another promising direction is exploring communication-avoiding variants of matrix multiplication. Algorithms that reduce the frequency or volume of MPI communication may significantly improve hybrid scaling efficiency, particularly on systems where GPU computation is substantially faster than network transfer. Investigating topology-aware rank placement, GPUDirect RDMA, and NCCL-based communication primitives may further reduce communication overhead for multi-GPU distributed workloads. These optimizations are increasingly important as modern supercomputers continue shifting toward GPU-dense architectures.

Future work should focus on: (1) overlapping MPI communication with CUDA computation using CUDA streams and non-blocking MPI calls to hide latency; (2) testing at larger matrix sizes ( $2^{16} \times 2^{16}$  and beyond) to characterize the distributed GPU scaling crossover point; (3) replacing naive CUDA kernels with cuBLAS-accelerated submatrix multiplication; (4) incorporating MPI I/O for checkpoint/restart capabilities using AiMOS NVMe storage; and (5) extending the weak scaling study to 256 or more ranks to assess behavior at supercomputing scale.

## References

- [1] L.E. Cannon, A cellular computer to implement the Kalman filter algorithm, Ph.D. thesis, Montana State University, 1969.
- [2] G.C. Fox et al., Solving Problems on Concurrent Processors, Prentice Hall, 1988.
- [3] R.A. van de Geijn and J. Watts, SUMMA: Scalable Universal Matrix Multiplication Algorithm, *Concurrency: Practice and Experience*, 9(4):255–274, 1997.
- [4] Message Passing Interface Forum, MPI: A Message-Passing Interface Standard Version 4.1, 2023.
- [5] NVIDIA Corporation, CUDA C++ Programming Guide, Version 12.0, 2023.
- [6] A. Grama, A. Gupta, G. Karypis, V. Kumar, *Introduction to Parallel Computing*, 2nd ed., Addison-Wesley, 2003.
- [7] J.J. Dongarra et al., The LINPACK Benchmark: Past, Present, and Future, *Concurrency and Computation: Practice and Experience*, 15(9):803–820, 2003.
- [8] S. Tomov, R. Nath, H. Ltaief, J. Dongarra, Dense Linear Algebra Solvers for Multicore with GPU Accelerators, IPDPS 2010.
- [9] IBM Research, POWER9 Processor User's Manual, 2018.
- [10] CCI RPI, DCS Supercomputer (AiMOS) Documentation, <https://docs.cci.rpi.edu>, 2024.
- [11] V. Volkov and J.W. Demmel, Benchmarking GPUs to Tune Dense Linear Algebra, SC '08: Proceedings of the 2008 ACM/IEEE Conference on Supercomputing, 2008.
- [12] J. Kurzak and J. Dongarra, Implementing Linear Algebra Routines on Multi-Core Processors, LAPACK Working Note 196, 2007.